

TC631 OPERATING SYSTEM

Task 1: Thread and Multithreading

Multithreading, Mutex, Semaphore, and Coroutine in C++

Azriel Fajar Wicaksono

NIM 672025121

Instructor: Theophilus Wellem

Faculty of Information Technology

Satya Wacana Christian University

2026

1. Introduction

A **thread** is a unit of execution in which instructions run one after another. Every program has at least one thread, the **main thread**, which runs the code inside `main()`. A program that starts extra threads next to the main thread is a **multithreaded application**. The threads can run in parallel, so the operating system can use more than one CPU core.

When threads share data, they must be **synchronized**, or the data can become wrong. This report shows four simple programs, one for each concept in the assignment:

Program	Concept	Scenario	Main C++ feature
<code>1_thread.cpp</code>	Multithreading	Two workers run at the same time as main	<code>std::jthread</code> , <code>join()</code>
<code>2_mutex.cpp</code>	Mutex	Two threads add to one shared counter	<code>std::mutex</code>
<code>3_semaphore.cpp</code>	Semaphore	Five cars share a parking lot with two spots	<code>std::counting_semaphore</code>
<code>4_coroutine.cpp</code>	Coroutine	A countdown that pauses and resumes	<code>co_yield</code> , <code>std::generator</code>

1.1 Environment

- Operating system: Windows 11
- Compiler: g++ (Rev13, Built by MSYS2 project) 15.2.0
- Language standard: C++20 for Programs 1 to 3, C++23 for Program 4 (`std::generator` was added in C++23)

Each program is compiled with `g++ -std=c++20 file.cpp -o file` (or `-std=c++23` for Program 4). The flag is required: without it, g++ uses C++17, where `std::jthread`, `std::counting_semaphore`, `std::format` and `std::generator` do not exist. All outputs in this report are real outputs from running the programs.

Programs 1 and 3 print with `std::cout << std::format(...)`. `std::format` builds the whole line first, so each line is written in one call. If a line were printed piece by piece with several `<<`, pieces from different threads could get mixed in the same line.

2. Program 1: Multithreading

2.1 Objective

Show that threads run at the same time. The main thread starts two worker threads, keeps running, and then waits for both workers to finish before the program ends.

2.2 Source Code

1_thread.cpp

```
1 // Program 1: Multithreading
2 // Two worker threads run at the same time as the main thread.
3 // Compile: g++ -std=c++20 1_thread.cpp -o 1_thread
4
5 #include <thread>
6 #include <iostream>
7 #include <format>
8 #include <chrono>
9
10 using namespace std::chrono_literals;
11
12 // The job each worker thread runs
13 void Worker(int id)
14 {
15     for (int step = 1; step <= 3; ++step)
16     {
17         std::cout << std::format("Worker {}: step {}\n", id, step);
18         std::this_thread::sleep_for(500ms); // pretend to do some work
19     }
20     std::cout << std::format("Worker {}: finished\n", id);
21 }
22
23 int main()
24 {
25     std::cout << "Main: starting 2 worker threads\n";
26
27     // Creating a jthread starts it immediately
28     std::jthread worker1(Worker, 1);
29     std::jthread worker2(Worker, 2);
30
31     std::cout << "Main: workers are running, main keeps going\n";
32
33     // Thread synchronization: main waits until both workers are done
34     worker1.join();
35     worker2.join();
36
37     std::cout << "Main: all workers finished, exiting\n";
38     return 0;
39 }
```

2.3 How the Program Works

1. **Lines 13–21, `Worker(int id)`**: the function that every worker thread runs. It prints three steps. After each step it sleeps for 500 ms (line 18) to act like it is doing real work. At the end it prints that it is finished.
2. **Lines 28–29**: creating a `std::jthread` object starts a new thread **immediately**. `worker1` runs `Worker(1)` and `worker2` runs `Worker(2)`. From this point there are three threads: main, worker 1 and worker 2.
3. **Line 31**: the main thread does not wait for the workers. It continues to its next line right away.
4. **Lines 34–35, `join()`**: the main thread blocks until `worker1` ends, then until `worker2` ends. This is the simplest form of **thread synchronization**: one thread waits on another. (A `std::jthread` also joins automatically when it is destroyed. The `join()` calls are written here to make the waiting visible.)

5. **Line 37:** this line only runs after both workers are done, so it is always the last line of the output.

2.4 Output

```
Terminal (PowerShell)

> g++ -std=c++20 1_thread.cpp -o 1_thread
> .\1_thread.exe
Main: starting 2 worker threads
Main: workers are running, main keeps going
Worker 1: step 1
Worker 2: step 1
Worker 1: step 2
Worker 2: step 2
Worker 1: step 3
Worker 2: step 3
Worker 1: finished
Worker 2: finished
Main: all workers finished, exiting
```

2.5 Analysis

- The steps of Worker 1 and Worker 2 are mixed together. Both threads are active at the same time, which shows that they run in parallel.
- The order of Worker 1 and Worker 2 inside the same step can change from run to run. The **operating system scheduler** decides which thread runs first. The program has no control over it.
- Each worker sleeps $3 \times 500 \text{ ms} = 1.5 \text{ s}$. If the workers ran one after another, the program would take about 3 s. The measured run time was **1.62 s** (1.5 s of work plus program start-up time), because both workers sleep at the same time.
- "Main: all workers finished" always comes last, because of the two `join()` calls.

3. Program 2: Mutex

3.1 Objective

Show what goes wrong when two threads write to the same shared variable without synchronization (a **race condition**), and how a **mutex** fixes it. Two threads each add 1 to a shared `counter` 1,000,000 times, so the correct result is 2,000,000.

3.2 Source Code

```
2_mutex.cpp

1 // Program 2: Mutex
2 // Two threads add to the same shared counter.
3 // Without a mutex the result is wrong. With a mutex it is correct.
4 // Compile: g++ -std=c++20 2_mutex.cpp -o 2_mutex
5 // (Do NOT add -O2. The optimizer can hide the bug in the "without mutex" test.)
6
7 #include <thread>
8 #include <mutex>
9 #include <iostream>
10
11 const int TIMES = 1000000;
12
13 int counter = 0; // shared variable, both threads use it
14 std::mutex counterMutex; // protects counter
15
16 // NOT safe: both threads can change counter at the same moment
17 void AddWithoutMutex()
18 {
19     for (int i = 0; i < TIMES; ++i)
20         counter++;
21 }
22
23 // Safe: only the thread holding the mutex can change counter
24 void AddWithMutex()
25 {
26     for (int i = 0; i < TIMES; ++i)
27     {
28         counterMutex.lock(); // acquire: other threads wait here
29         counter++;           // critical section
30         counterMutex.unlock(); // release: next thread can enter
31     }
32 }
33
34 int main()
35 {
36     // Test 1: without mutex
37     counter = 0;
38     std::jthread a(AddWithoutMutex);
39     std::jthread b(AddWithoutMutex);
40     a.join();
41     b.join();
42     std::cout << "Without mutex: " << counter << " (expected " << TIMES * 2 << ")\n";
43
44     // Test 2: with mutex
45     counter = 0;
46     std::jthread c(AddWithMutex);
47     std::jthread d(AddWithMutex);
48     c.join();
49     d.join();
50     std::cout << "With mutex: " << counter << " (expected " << TIMES * 2 << ")\n";
51
52     return 0;
53 }
```

3.3 How the Program Works

1. **Lines 13–14:** `counter` is a global variable, so both threads share it. `counterMutex` is the mutex that protects it.
2. **Lines 17–21, `AddWithoutMutex()`** : adds 1 to `counter` 1,000,000 times with no protection.
3. **Lines 24–32, `AddWithMutex()`** : the same loop, but each `counter++` is inside the mutex:
 - Line 28, `lock()` : the thread **acquires** the mutex. If another thread already holds it, this thread waits (the OS puts it to sleep) until the mutex is released.
 - Line 29, `counter++` : the **critical section**. Only one thread can be here at a time.
 - Line 30, `unlock()` : the thread **releases** the mutex, so a waiting thread can enter.
4. **Lines 37–42, Test 1:** two threads run `AddWithoutMutex` at the same time. Main joins both and prints the result.
5. **Lines 45–50, Test 2:** the same test with `AddWithMutex`.

3.4 Why the Result Is Wrong Without a Mutex

`counter++` looks like one operation, but the CPU does it in three steps: **read** the value from memory, **add** 1, and **write** it back. If two threads do these steps at the same moment, one addition can be lost:

Time	Thread A	Thread B	counter in memory
1	read counter (10)		10
2		read counter (10)	10
3	add 1, write 11		11
4		add 1, write 11	11 (should be 12)

This happens many times during 2,000,000 additions, so the final value is too small. With the mutex, Thread B cannot start its read until Thread A has finished its write and unlocked, so no addition is lost.

3.5 Output (three runs)

```
Terminal (PowerShell)

> g++ -std=c++20 2_mutex.cpp -o 2_mutex
> .\2_mutex.exe
Without mutex: 1151583 (expected 2000000)
With mutex:    2000000 (expected 2000000)
> .\2_mutex.exe
Without mutex: 1149186 (expected 2000000)
With mutex:    2000000 (expected 2000000)
> .\2_mutex.exe
Without mutex: 1286422 (expected 2000000)
With mutex:    2000000 (expected 2000000)
```

3.6 Analysis

- **Without mutex:** the result is always below 2,000,000 and different in every run. The result depends on how the OS schedules the two threads, so it cannot be predicted.
- **With mutex:** the result is exactly 2,000,000 every time.
- **Ownership:** a mutex has an owner. Only the thread that locked it may unlock it. This is the main difference from a semaphore (Program 3).

- The program must be compiled **without** `-O2`. When I tested with `-O2`, the compiler optimized the loop and the result without a mutex was also 2,000,000, which hides the race condition. The bug is still in the code. It just does not show.
- In real programs, `std::lock_guard<std::mutex>` is usually used instead of calling `lock()` and `unlock()` by hand, because it unlocks automatically, even if an error happens. Here the calls are written out so each step is visible.

4. Program 3: Semaphore

4.1 Objective

Show how a **counting semaphore** limits how many threads can use a shared resource at the same time. A parking lot has only **2 spots**, but **5 cars** (5 threads) want to park. The semaphore makes sure that no more than 2 cars are parked at once.

4.2 Source Code

```
3_semaphore.cpp

1 // Program 3: Semaphore
2 // A parking lot has only 2 spots, but 5 cars (threads) want to park.
3 // The semaphore lets at most 2 cars in at the same time.
4 // Compile: g++ -std=c++20 3_semaphore.cpp -o 3_semaphore
5
6 #include <thread>
7 #include <semaphore>
8 #include <iostream>
9 #include <format>
10 #include <chrono>
11 #include <vector>
12
13 using namespace std::chrono_literals;
14
15 // Counting semaphore, starts with 2 free spots
16 std::counting_semaphore<2> parkingSpots(2);
17
18 void Car(int id)
19 {
20     std::cout << std::format("Car {}: waiting for a spot\n", id);
21
22     parkingSpots.acquire(); // take a spot (count - 1). If count is 0, wait here.
23     std::cout << std::format("Car {}: PARKED\n", id);
24
25     std::this_thread::sleep_for(1s); // stay parked for a while
26
27     std::cout << std::format("Car {}: leaving\n", id);
28     parkingSpots.release(); // give the spot back (count + 1)
29 }
30
31 int main()
32 {
33     std::vector<std::jthread> cars;
34     for (int id = 1; id <= 5; ++id)
35     {
36         cars.emplace_back(Car, id);
37         std::this_thread::sleep_for(50ms); // cars arrive one after another
38     }
39
40     for (auto& car : cars)
41         car.join();
42
43     std::cout << "All cars are done\n";
44     return 0;
45 }
```

4.3 How the Program Works

1. **Line 16:** `std::counting_semaphore<2> parkingSpots(2)` creates a semaphore. It is an integer counter that starts at 2 (two free spots). The template value `<2>` is the maximum the counter can reach.
2. **Lines 18–29, `Car(int id)`:** the function each car thread runs.

- Line 22, `acquire()` (wait): if the counter is above 0, it is decreased by 1 and the car parks. If the counter is 0, the thread **blocks** until another thread releases a spot.
- Line 25: the car stays parked for 1 second.
- Line 28, `release()` (signal): the counter is increased by 1. If a car is waiting, one waiting thread is woken up and can park.

3. **Lines 33–38:** main creates 5 car threads, 50 ms apart, and stores them in a vector.

4. **Lines 40–41:** main joins every car thread, then prints the final line.

4.4 Example Timeline of the Counter

Time	Event	Counter
0.00 s	Car 1 calls <code>acquire()</code> and parks	2 → 1
0.05 s	Car 2 calls <code>acquire()</code> and parks	1 → 0
0.10–0.20 s	Cars 3, 4, 5 call <code>acquire()</code> and block	0
1.00 s	Car 1 calls <code>release()</code> , one waiting car wakes up and parks	0 → 1 → 0
1.05 s	Car 2 calls <code>release()</code> , another waiting car parks	0 → 1 → 0
2.00 s and later	The last car parks, then all cars leave	back to 2

4.5 Output

```
Terminal (PowerShell)
> g++ -std=c++20 3_semaphore.cpp -o 3_semaphore
> .\3_semaphore.exe
Car 1: waiting for a spot
Car 1: PARKED
Car 2: waiting for a spot
Car 2: PARKED
Car 3: waiting for a spot
Car 4: waiting for a spot
Car 5: waiting for a spot
Car 1: leaving
Car 3: PARKED
Car 2: leaving
Car 4: PARKED
Car 3: leaving
Car 5: PARKED
Car 4: leaving
Car 5: leaving
All cars are done
```

4.6 Analysis

- Cars 1 and 2 park right away. Cars 3, 4 and 5 print "waiting" and then block, because the counter is 0.
- A new car parks only right after another car leaves. At no point are more than 2 cars parked at the same time. (The build script that produced this report checks this in the output automatically.)
- A semaphore does **not** guarantee first come, first served. When a spot is released, any waiting car can get it, depending on which thread the OS wakes up. So the parking order of cars 3, 4 and 5 can change between runs.
- **No ownership:** unlike a mutex, a semaphore does not remember which thread took a spot. Any thread may call `release()`.

- A semaphore that starts at 1 is a **binary semaphore** (`std::binary_semaphore`). It lets only one thread in, like a mutex, but still without ownership.
- Real uses: limiting a database connection pool, or limiting how many downloads or HTTP requests run at the same time.

5. Program 4: Coroutine

5.1 Objective

Show that a **coroutine** is a function that can **pause** in the middle and **resume** later from the same place. The whole program runs in **one thread**. The coroutine and `main` take turns, and the coroutine itself decides when to pause (**cooperative multitasking**).

5.2 Source Code

```
4_coroutine.cpp

1 // Program 4: Coroutine
2 // A coroutine is a function that can pause (co_yield) and continue later.
3 // Everything here runs in ONE thread. The coroutine and main take turns.
4 // Compile: g++ -std=c++23 4_coroutine.cpp -o 4_coroutine
5 // (std::generator is a C++23 helper that makes coroutines easy to write)
6
7 #include <generator>
8 #include <iostream>
9
10 // A coroutine that counts down from "start"
11 std::generator<int> Countdown(int start)
12 {
13     std::cout << " [coroutine] started\n";
14     for (int n = start; n > 0; --n)
15     {
16         std::cout << " [coroutine] giving " << n << ", then pausing\n";
17         co_yield n; // PAUSE here and send n back to main
18         std::cout << " [coroutine] resumed\n";
19     }
20     std::cout << " [coroutine] finished\n";
21 }
22
23 int main()
24 {
25     std::cout << "[main] calling the coroutine\n";
26
27     // Each loop step resumes the coroutine until its next co_yield
28     for (int value : Countdown(3))
29     {
30         std::cout << "[main] received " << value << ", doing my own work\n";
31     }
32
33     std::cout << "[main] done\n";
34     return 0;
35 }
```

5.3 How the Program Works

1. **Line 11:** `Countdown` returns `std::generator<int>` and its body uses `co_yield`, so the compiler turns it into a coroutine. The keywords `co_yield`, `co_await` and `co_return` come from C++20. The ready-made `std::generator` type comes from C++23. It handles the setup code that C++20 would otherwise need, which keeps the program short.
2. **Line 28:** calling `Countdown(3)` does **not** run the body yet. The body starts only when the `for` loop asks for the first value. This is why "[main] calling the coroutine" is printed before "[coroutine] started".
3. **Line 17, `co_yield n`:** the coroutine sends `n` to main and **pauses right there**. Its local variables (`n`, `start`) and its position are saved in the coroutine frame.
4. **Line 30:** main receives the value and does its own work.

5. When the loop asks for the next value, the coroutine **resumes** on line 18, the line right after `co_yield`, not from the start of the function.
6. When the coroutine reaches the end of its body (line 20), the `for` loop in main ends.

5.4 Output

```
Terminal (PowerShell)
> g++ -std=c++23 4_coroutine.cpp -o 4_coroutine
> .\4_coroutine.exe
[main] calling the coroutine
[coroutine] started
[coroutine] giving 3, then pausing
[main] received 3, doing my own work
[coroutine] resumed
[coroutine] giving 2, then pausing
[main] received 2, doing my own work
[coroutine] resumed
[coroutine] giving 1, then pausing
[main] received 1, doing my own work
[coroutine] resumed
[coroutine] finished
[main] done
```

5.5 Analysis

- Control goes back and forth: coroutine, main, coroutine, main. This is visible from the indented "[coroutine]" lines and the "[main]" lines.
- "[coroutine] resumed" appears right after main's line, which shows that the coroutine continues from where it paused, with its state (the value of `n`) kept.
- The output is exactly the same on every run. There is only one thread, so the OS scheduler never interrupts the coroutine. It only stops when it reaches `co_yield`. Threads work the other way: the OS can interrupt them at any moment (**preemptive scheduling**), which is why Programs 1 and 3 can print in a different order each run.
- No mutex is needed, because the coroutine and main never run at the same time.
- Coroutines are cheaper than threads, because pausing and resuming is only a function jump, with no OS thread switch. This is why they are used for many I/O tasks at once, like web servers and network sockets, through `async` / `await` patterns.

6. Comparison and Conclusion

	Thread	Mutex	Semaphore	Coroutine
What it is	An independent path of execution	A lock that lets 1 thread in at a time	A counter that lets up to N threads in	A function that can pause and resume
Runs in parallel?	Yes	Synchronization tool	Synchronization tool	No, one thread takes turns
Who switches?	The OS (preemptive)	–	–	The code itself at <code>co_yield</code> (cooperative)
Ownership	–	Yes, only the locking thread may unlock	No, any thread may release	–
C++ feature	<code>std::jthread</code>	<code>std::mutex</code>	<code>std::counting_semaphore</code>	<code>co_yield</code> , <code>std::generator</code>
Shown in	Program 1	Program 2	Program 3	Program 4

The four programs show that multithreading lets a program do several things at the same time, but the order of execution is decided by the operating system and changes between runs. When threads share data, synchronization is required. A **mutex** protects a critical section so that only one thread uses it at a time. A **semaphore** allows a fixed number of threads to use a limited resource. A **coroutine** is different from both: it does not create new threads. It runs in one thread and pauses on its own, so its execution order is always predictable.

The result that surprised me most was Program 2. A single line like `counter++` looks safe, but in the three runs shown in Section 3.5 it lost between 713,578 and 850,814 additions when two threads ran it at the same time. The number was also different every run. This showed me why shared data between threads must always be protected.

References

1. S. Rao, *Sams Teach Yourself C++ in One Hour a Day*, 9th ed. Pearson, 2022.
2. T. Wellem, "TC631 Operating System, Assignment #1: Thread and Multithreading," lecture slides, Satya Wacana Christian University, 2026.
3. cppreference.com, "`std::jthread`," "`std::mutex`," "`std::counting_semaphore`," and "`std::generator`." Available: <https://en.cppreference.com/w/cpp/thread> and <https://en.cppreference.com/w/cpp/coroutine/generator>